

MODUL PRAKTIKUM CAPSTONE (JARINGAN & KONKURENSI)

TOPIK: IMPLEMENTASI MULTI-THREADED WEB SERVER BERBASIS POSIX SOCKET

Mata Kuliah: Sistem Operasi (INF-201) – Alokasi Waktu: 200 Menit – Bobot Evaluasi: 10%

Program Studi Informatika – Fakultas Teknik – Universitas Bengkulu

1. CAPAIAN PEMBELAJARAN MODUL

Setelah menyelesaikan Modul Capstone paruh pertama ini, mahasiswa diharapkan mampu:

1. Memahami konsep **System Calls** terkait jaringan (Network Sockets: `socket`, `bind`, `listen`, `accept`).
2. Mengimplementasikan protokol aplikasi sederhana (HTTP/1.1) di atas lapisan transport (TCP).
3. Merancang arsitektur konkurensi kinerja tinggi menggunakan *Thread Pool* dengan library POSIX Threads (`pthread`).
4. Menganalisis masalah kondisi balapan (*race condition*) dalam buffer jaringan bersama.

2. LANDASAN TEORI RINGKAS

Sebuah Web Server konvensional memproses **request** dari *web browser* melalui **Network Socket**. Jika server bersifat **single-threaded**, ia hanya bisa melayani satu permintaan dalam satu waktu (permintaan klien kedua akan terblokir menunggu klien pertama selesai).

Untuk mengatasi ini, Sistem Operasi modern menggunakan pendekatan **Multi-threading**. Salah satu model terbaik adalah **Thread Pool**: Server membuat sekumpulan *worker threads* saat inisialisasi. Setiap kali **request** baru datang, **request** tersebut dimasukkan ke dalam antrean (Buffer). **Worker thread** yang menganggur akan mengambil **request** dari antrean tersebut dan melayaninya. Hal ini merupakan implementasi dunia nyata dari masalah **Producer-Consumer**.

3. KODE ACUAN DASAR (SINGLE-THREADED HTTP SERVER)

Berikut adalah kerangka server TCP statis yang hanya melayani satu permintaan pada satu waktu (`webserver_single.c`).

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <arpa/inet.h>
6  #include <pthread.h>
7
8  #define PORT 8080
9  #define BUFFER_SIZE 1024
10
11 // Fungsi untuk melayani koneksi klien
12 void *handle_client(void *client_socket) {
13     int sock = *((int *)client_socket);
14     free(client_socket);
15     char buffer[BUFFER_SIZE];
16 }
```

```

17 // Membaca request HTTP dari browser
18 read(sock, buffer, BUFFER_SIZE - 1);
19 printf("Request diterima:\n%s\n", buffer);
20
21 // Mengirim balasan HTTP sederhana
22 char *response = "HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\n\r\n"
23                 "Halo dari Sistem Operasi Universitas Bengkulu!";
24 write(sock, response, strlen(response));
25
26 // Tutup socket
27 close(sock);
28 return NULL;
29 }
30
31 int main() {
32     int server_fd, *new_sock;
33     struct sockaddr_in address;
34     int addrlen = sizeof(address);
35
36     // 1. Membuat file descriptor untuk TCP Socket
37     if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
38         perror("Socket gagal"); exit(EXIT_FAILURE);
39     }
40
41     address.sin_family = AF_INET;
42     address.sin_addr.s_addr = INADDR_ANY;
43     address.sin_port = htons(PORT);
44
45     // 2. Binding socket ke PORT 8080
46     if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
47         perror("Bind gagal"); exit(EXIT_FAILURE);
48     }
49
50     // 3. Listening untuk koneksi masuk
51     if (listen(server_fd, 3) < 0) {
52         perror("Listen gagal"); exit(EXIT_FAILURE);
53     }
54
55     printf("Web Server berjalan di http://localhost:%d\n", PORT);
56
57     // Loop penerimaan koneksi
58     while(1) {
59         new_sock = malloc(sizeof(int));
60         *new_sock = accept(server_fd, (struct sockaddr *)&address, (
61             socklen_t*)&addrlen);
62
63         // TODO (Di sinilah letak masalah Single-Thread):
64         // Panggilan ini akan memblokir main loop!
65         handle_client(*new_sock);
66     }
67
68     return 0;
69 }

```

Listing 1: Kerangka Single-Threaded Server

4. TUGAS CAPSTONE: THREAD POOL & MUTEX SYNCHRONIZATION

Tugas Anda adalah memodifikasi kode di atas menjadi `webserver_multi.c`. Jangan membuat *thread* baru untuk setiap klien (karena akan membanjiri OS jika diserang jutaan *request*). Sebaliknya, terapkan **Thread Pool Architecture**:

1. Buat sebuah *array/queue* global berukuran tetap (misal: 100 elemen) untuk menyimpan *socket file descriptors* klien yang masuk (*Connection Buffer*).
2. Buat sejumlah 10 *Worker Threads* di awal program (fungsi `main`).
3. Thread `main` akan bertindak sebagai **Producer**: Ia hanya menerima koneksi via `accept()` lalu memasukkannya ke dalam *Connection Buffer*.
4. Kesepuluh *Worker Threads* akan bertindak sebagai **Consumer**: Mereka terus-menerus mencoba mengambil *socket* dari **Connection Buffer** lalu menjalankan `handle_client()`.
5. Lindungi *Connection Buffer* menggunakan `pthread_mutex_t` dan gunakan `pthread_cond_t` agar **Worker Threads** tertidur (sleep) saat **Buffer** kosong.

5. PENGUJIAN BEBAN (LOAD TESTING)

Setelah server **multi-threaded** Anda berjalan, kompilasi dan serang server Anda menggunakan **benchmarking tool** seperti Apache Bench (`ab`) untuk menguji keandalan konkurensinya:

```
1 # Kompilasi dengan ThreadSanitizer
2 $ gcc -Wall -pthread -fsanitize=thread webserver_multi.c -o webserver_multi
3 $ ./webserver_multi
4
5 # Di terminal lain, kirim 10.000 request dengan konkurensi 100 klien!
6 $ ab -n 10000 -c 100 http://localhost:8080/
```

Jika arsitektur *Thread Pool* dan *Mutex* Anda salah, server akan *crash*, menghasilkan hasil giberish, atau *ThreadSanitizer* akan mencetak log bahaya **Data Race**.